

OCR Baseline Notebook — Implementation Specification

Project: Scaling laws for OCR under label noise **Purpose of this document:** a complete build spec for the first notebook. Hand this to a code generator, or implement it cell by cell yourself. **Version 2** — supersedes v1. Changes: dataset-vs-scaling-axis rule (§3), confirmed XML formats per dataset (§4), expected format for inspection (§5), new §18 on the size limitation and migration path. **Target environment:** Google Colab, single T4 GPU, PyTorch.

0. What this notebook is and is not

It is a reproducible pipeline that trains a small OCR model from scratch on FID4SA ground-truth data, with two knobs exposed as configuration: **how much training data to use**, and **how much label noise to inject**.

It is not an attempt to build an accurate OCR system. Do not add tricks that improve accuracy at the cost of reproducibility.

The three design constraints, and why they exist

Constraint	Reason
Train from scratch. No pretrained weights.	The research question is how error scales with <i>training data seen</i> . A pretrained model has already seen an unknown, enormous amount of data, which destroys the x-axis.
Every run writes one row to a results file.	A scaling law is fitted from many runs. If results live only in notebook output, they are lost on disconnect.
Every source of randomness is seeded and logged.	Runs must be comparable across data sizes and noise rates. An unseeded run is a wasted run.

Success criterion for the first run

A model that trains, a loss that decreases, and a validation CER below roughly 30%. Below 10% is good. Do **not** chase the ~2.29% CER reported by the Heidelberg team using Transkribus — that is a mature production tool, and matching it is not the goal.

1. Notebook structure

Fourteen sections, in this order. Each is one or more cells.



1. Setup and dependencies
2. Configuration block
3. Data download
4. XML format inspection <- STOP AND READ OUTPUT
5. Line extraction (XML -> line images + manifest)
6. Text normalisation and vocabulary
7. Label noise injection
8. Dataset and DataLoader
9. Model (CRNN)
10. Decoding and metrics
11. Training loop
12. Evaluation and sample predictions
13. Results logging
14. Sanity checks

2. Section 1 — Setup and dependencies



Check GPU is available; print device name. Abort with a clear message if CPU only.

Mount Google Drive at `/content/drive`.

Create directory structure under Drive:

```
<PROJECT_ROOT>/
  raw/      downloaded archives
  lines/    extracted line images
  manifests/ CSV manifests
  checkpoints/
  results/  results.csv, run configs
```

Set all seeds: random, numpy, torch, torch.cuda.

Set `torch.backends.cudnn.deterministic = True`, `benchmark = False`.

Packages: `torch`, `torchvision`, `Pillow`, `pandas`, `numpy`, `lxml`, `editdistance`, `tqdm`, `matplotlib`. Colab has most; `editdistance` and `lxml` may need `pip install`.

Drive I/O warning: reading thousands of small files directly from Drive is extremely slow. Keep the *archive* and *outputs* on Drive, but copy the extracted line images to `/content/lines/` (local disk) at the start of each session and train from there.

3. Section 2 — Configuration block

One dictionary or dataclass holding every knob. Nothing configurable may be hard-coded elsewhere in the notebook.



```
python
CONFIG = {
    # --- experiment identity ---
    "run_name":    "baseline_d100_n00", # auto-generate from the fields below
    "seed":       42,

    # --- data ---
    "dataset":     "devanagari",        # devanagari | malayalam | bengali
    "data_fraction": 1.0,                # THE SCALING-LAW AXIS. 0.125, 0.25, 0.5, 1.0
    "max_train_lines": None,             # absolute cap, alternative to fraction
    "val_split":   0.10,
    "test_split":  0.10,

    # --- label noise ---
    "noise_type":   "none",              # none | char_flip | line_swap
    "noise_rate":   0.0,                 # THE NOISE AXIS. 0.0, 0.1, 0.2, 0.4
    "noise_seed":   1234,                # SEPARATE from training seed, see 7.3

    # --- image preprocessing ---
    "img_height":   32,
    "img_max_width": 512,
    "grayscale":    True,

    # --- model ---
    "cnn_width_mult": 1.0,               # MODEL-SIZE AXIS for later runs
    "rnn_hidden":    256,
    "rnn_layers":    2,
    "dropout":       0.1,

    # --- training ---
    "batch_size":    32,
    "epochs":        60,
    "lr":            3e-4,
    "optimizer":     "adam",
    "weight_decay":  0.0,
    "scheduler":     "cosine",
    "grad_clip":     5.0,
    "early_stop_patience": 12,
    "augment":       False,              # keep False for scaling runs
}
```

Rules:

`dataset` is a **task identifier, not a scaling axis**. Each script has a different alphabet, a different vocabulary size and a different intrinsic difficulty, so runs on different scripts are points on *different* curves, never on the same one. Never pool them into a single fit. Comparing exponents *across* scripts is a legitimate separate question; treat it as such.

`run_name` must encode dataset, data_fraction, noise_type, noise_rate and seed, so checkpoints never overwrite each other.

Dump `CONFIG` to `results/<run_name>_config.json` at the start of every run.

`augment` defaults to **False**. Augmentation changes the effective dataset size, which contaminates the very quantity being measured. Only turn it on for a deliberate side experiment.

4. Section 3 — Data download

Source: heiDATA, Heidelberg University. Collection: *Ground truth data for HTR on South Asian Scripts* (FID4SA-GT).

FID4SA-GT is a **collection of separate datasets**, not one dataset. Each has its own DOI, contributors and file format, and is downloaded individually.

Dataset	Persistent ID	Reported size	XML format
Printed Devanagari	<code>doi:10.11588/data/EG0KEI</code>	~220 pages, ~27,000 words	ALTO
Printed Malayalam	<code>doi:10.11588/data/L2KRZO</code>	43 pages, ~9,000 words	PAGE and ALTO
Bengali (historical printed)	<code>doi:10.11588/data/AIQSXL</code> —		republished with corrected line coordinates

The collection also holds **handwritten** manuscript data (e.g. Nepalese Pracalit). Do not mix handwritten and printed material — they are different visual regimes and pooling them contaminates any scaling curve.

Start with Devanagari — it is the largest and gives the most stable first result. Its source material is letterpress printing from the Naval Kishore Press in Lakhnau, late 19th to early 20th century, covering Hindi, Sanskrit, Braj Bhasha and Awadhi.

The datasets are distributed as archives containing **JPG images and ALTO or PAGE XML files**.

☐ Download the archive manually through the browser and place it in Drive under raw/, OR fetch it via the Dataverse API if a direct download URL is available. Unzip to raw/<dataset>/. Print a directory tree, two levels deep, plus counts of *.jpg and *.xml.

Assert early: the number of XML files should be broadly comparable to the number of images. A large mismatch means the archive layout differs from expectation, and the parser will need adjusting.

5. Section 4 — XML format inspection

This section exists because the parser cannot be written correctly until the schema is confirmed. Run it and read the output before writing Section 5.

Expected result for the Devanagari set: **ALTO**. Treat that as a hypothesis to confirm, not a fact — if the inspection reports PAGE, believe the inspection and adjust.

☐

Pick the first XML file.
Print its first 80 lines verbatim.
Parse it and print the root tag and all namespace declarations.
Detect the format:

- root tag contains 'alto' -> ALTO
- root tag contains 'PcGts' -> PAGE
- otherwise -> unknown, print all distinct tags and stop

Count TextLine elements in this file.
Print the first three TextLine elements in full.

What the two formats look like

ALTO — geometry as rectangles, text in an attribute:

☐

```
xml
<TextLine ID="line_1" HPOS="120" VPOS="340" WIDTH="1500" HEIGHT="60">
  <String CONTENT="अथ प्रथमोऽध्यायः" .../>
</TextLine>
```

Line text = the `CONTENT` attributes of child `String` elements, joined with spaces (respect any `SP` elements).

PAGE — geometry as a polygon, text in a child element:

☐

```
xml
<TextLine id="line_1">
  <Coords points="120,340 1620,340 1620,400 120,400"/>
  <TextEquiv><Unicode>अथ प्रथमोऽध्यायः</Unicode></TextEquiv>
</TextLine>
```

Line text = the `<Unicode>` element inside `<TextEquiv>`.

Namespaces are mandatory in both formats. Use `lxml` and pass an explicit namespace map, or match on local names with `{*}TextLine`. Plain tag matching will silently return nothing.

Both may be present. The Bengali dataset was republished with corrected line coordinates, and some entries ship both PAGE and ALTO. Prefer PAGE when both exist — polygons are more precise.

6. Section 5 — Line extraction

Convert pages into individual line images plus a manifest.

6.1 Functions

```
python
detect_format(xml_path) -> "alto" | "page"

parse_alto(xml_path) -> list[LineRecord]
parse_page(xml_path) -> list[LineRecord]

LineRecord = {
    "page_id": str,
    "line_id": str,
    "bbox": (x0, y0, x1, y1), # ints, axis-aligned
    "polygon": list[(x,y)] | None,
    "text": str,
}

extract_lines(raw_dir, out_dir) -> pandas.DataFrame
```

6.2 Cropping rules

For PAGE polygons, take the **axis-aligned bounding box** of the polygon. Do not attempt polygon masking or dewarping in v1.

Pad the box by 2 pixels on each side, then clip to image bounds.

Convert to grayscale on save.

Save as PNG (lossless) named `<page_id>_<line_id>.png` in `lines/<dataset>/`.

6.3 Filtering — drop a line if any of these hold

Condition	Reason
text is empty or whitespace only	no target
height < 8 px or width < 16 px	degenerate crop
bbox falls outside the image	bad coordinates
aspect ratio (w/h) > 60	almost certainly a merged multi-line region

Condition	Reason
text length > 200 characters	same

Log every drop with its reason, and print a summary count. If more than ~10% of lines are dropped, the parser is probably wrong — investigate rather than proceeding.

6.4 Manifest

Write `manifests/<dataset>_lines.csv`:

column	notes
<code>line_path</code>	relative path to the PNG
<code>page_id</code>	needed for the split, see below
<code>line_id</code>	
<code>text</code>	the clean ground-truth transcription
<code>width</code> , <code>height</code>	original crop size
<code>n_chars</code>	after normalisation

6.5 Splitting — split by PAGE, not by line

Lines from the same page share typeface, ink, scan quality and page-level artefacts. Splitting randomly at line level puts near-identical lines in both train and test, which **leaks** and inflates the score.

Shuffle the list of unique <code>page_ids</code> with the fixed seed. Assign whole pages to train / val / test in 80 / 10 / 10 proportion. Assert the three page sets are disjoint. Print line counts per split.
--

Expected order of magnitude for Devanagari: roughly 3,000–8,000 lines. Record the real number — it directly limits how far the scaling curve can extend.

7. Section 6 — Text normalisation and vocabulary

7.1 Normalisation, applied before anything else touches the text

1. Unicode NFC normalisation.
2. Collapse runs of whitespace to a single space; strip leading/trailing space.
3. Optionally strip zero-width joiner / non-joiner (U+200D / U+200C) — decide, log the decision, apply consistently.

NFC normalisation is not optional for Indic scripts. The same visible Devanagari cluster can be encoded as different codepoint sequences. Without normalisation the vocabulary inflates, identical strings compare as unequal, and CER is computed against an inconsistent target.

7.2 Vocabulary



Build the charset from the TRAINING SPLIT ONLY, after normalisation.

Reserve index 0 for the CTC blank token.

Characters get indices 1..V.

Save to manifests/<dataset>_vocab.json.

Print V, and print the full sorted character list for visual inspection.

Inspect that printed list. Expect roughly 100–150 characters for Devanagari: consonants, independent vowels, matras, virama, anusvara, visarga, danda, digits, punctuation. Stray Latin letters or control characters indicate transcription artefacts worth cleaning.

Any character appearing in val/test but not in train must be mapped to a single `<unk>` index rather than crashing.

8. Section 7 — Label noise injection

This is the component the whole research project depends on. Build it carefully and test it in isolation.

8.1 Interface



```
python
inject_noise(df, noise_type, noise_rate, seed) -> (df_noisy, noise_mask)
```

Operates **only on the training split**. Validation and test labels stay clean, always. Measuring against corrupted labels is meaningless.

Returns a copy. Never mutates the input.

`noise_mask` is a boolean array, one entry per training line, `True` if that line's label was altered.

8.2 The two noise models

`char_flip` — **symmetric character-level noise**. For each character independently, with probability `noise_rate`, replace it with a character drawn uniformly from the vocabulary excluding itself and the blank.

This is a symmetric noise transition matrix over the alphabet: diagonal `1 - noise_rate`, off-diagonal `noise_rate / (V - 1)`. State it that way in the paper.

`line_swap` — **correspondence-style noise**. Select `noise_rate` of the training lines at random and permute their labels among the selected subset (a derangement, so no line keeps its own label).

These two are different problems and must not be mixed. Under `char_flip` a corrupted label remains mostly correct, so the model still receives usable signal and the per-sample loss rises only moderately. Under `line_swap` the label is entirely wrong and the loss is large. Co-teaching's small-loss criterion assumes something close to the second case. Run whichever Vinod specifies, but implement both, and never average results across them.

8.3 Seeding

`noise_seed` is deliberately **separate** from the training `seed`. This lets you re-run the same corrupted dataset under different training seeds, which is how you separate noise-induced variance from optimisation variance.

8.4 Persist the corruption

Save `results/<run_name>_noise_mask.npy` alongside the manifest, plus a CSV of the first 20 corrupted lines showing original and corrupted text side by side.

You will need the mask later to check whether co-teaching actually selects the clean samples. Without it that analysis is impossible, and regenerating it after the fact is error-prone.

8.5 Unit test — write this, do not skip it

☐

At `noise_rate = 0.0`: `df_noisy` is identical to `df`, mask is all False.
At `noise_rate = 1.0` (`char_flip`): no character survives unchanged.
At `noise_rate = 0.2` (`char_flip`): measured fraction of altered characters is 0.2 ± 0.02 .
At `noise_rate = 0.2` (`line_swap`): exactly 20% of rows have a different label, and no swapped row kept its own.
Same seed twice -> byte-identical output.

9. Section 8 — Dataset and DataLoader

9.1 Image preprocessing

☐

Load PNG as grayscale.
Resize so `height == img_height (32)`, preserving aspect ratio.
If resulting width > `img_max_width (512)`, resize down to `img_max_width` (ignore aspect).
Convert to float tensor, scale to `[0,1]`, normalise with mean 0.5 std 0.5.
Do NOT pad here — pad in the collate function.

9.2 Collate function for CTC

CTC needs concatenated targets plus explicit lengths.

☐

images: pad each to the max width in the batch (pad value = 1.0, i.e. white)
 -> tensor (B, 1, H, W_max)
input_lengths: downsampled width per sample (see 10.2), NOT W_max
targets: all label index sequences concatenated into one 1-D tensor
target_lengths: length of each label sequence

Sorting the batch by width descending reduces padding waste. Optional.

9.3 The CTC length constraint — the single most common silent failure

CTC requires, for every sample:

☐

input_length >= target_length

and in practice a comfortable margin, because repeated adjacent characters need a blank between them.

If this is violated, the loss for that sample is infinite. With `zero_infinity=True` those samples are silently dropped from the gradient and **training appears to work while learning nothing from the longest lines**.

Required check, run once before training:

☐

For every training sample, compute `downsampled_width` and `len(target)`.
Report: count and percentage where `downsampled_width < target_length * 1.2`
If that percentage exceeds ~1%, either raise `img_max_width`, reduce horizontal downsampling in the CNN, or tighten the max-length filter in 6.3.
Print the worst 10 offenders with their text.

10. Section 9 — Model (CRNN)

10.1 Architecture

Input `(B, 1, 32, W)`.

#	Layer	Output channels	Notes
1	Conv 3x3, s1, p1 + BN + ReLU	64	
2	MaxPool 2x2	64	H 32→16, W /2
3	Conv 3x3, s1, p1 + BN + ReLU	128	
4	MaxPool 2x2	128	H 16→8, W /2
5	Conv 3x3, s1, p1 + BN + ReLU	256	
6	Conv 3x3, s1, p1 + BN + ReLU	256	
7	MaxPool (2,1), stride (2,1)	256	H 8→4, W unchanged
8	Conv 3x3, s1, p1 + BN + ReLU	512	
9	MaxPool (2,1), stride (2,1)	512	H 4→2, W unchanged
10	Conv 2x2, s1, p0 + BN + ReLU	512	H 2→1

Output $(B, 512, 1, W/4)$. Squeeze height, permute to $(W/4, B, 512)$.

#	Layer
11	BiLSTM, hidden 256, 2 layers, dropout 0.1
12	Linear 512 → V+1

Total horizontal downsampling: /4. A 512-px-wide image yields 128 timesteps. This is what `input_lengths` must report: `ceil(original_width / 4)` after the height resize.

The pools at steps 7 and 9 deliberately collapse height only. Downsampling width further would break the CTC length constraint on long lines.

10.2 Model size as an experimental axis

Apply `cnn_width_mult` to every channel count and scale `rnn_hidden` proportionally. For later scaling-law runs use multipliers such as `[0.25, 0.5, 1.0, 2.0]`.

Print and log the total parameter count for every run. That number is the model-size axis; without it, model-scaling curves cannot be plotted.

11. Section 10 — Decoding and metrics

11.1 Greedy CTC decode

☐

argmax over the vocabulary at each timestep
collapse consecutive duplicate indices
remove all blank (index 0) entries
map indices back to characters

Do not implement beam search in v1. It improves accuracy but adds a confound: any change in decoding affects all runs and obscures the scaling signal.

11.2 Metrics

☐

$$\text{CER} = \frac{\text{sum}(\text{edit_distance}(\text{pred}, \text{truth}))}{\text{sum}(\text{len}(\text{truth}))}$$
 over the whole split
WER = same, computed over whitespace-split tokens

Aggregate at **corpus level** — total edits divided by total reference characters. Do **not** average per-line CER; short lines then dominate and the number is not comparable to published figures.

Use `editdistance.eval`. Compare NFC-normalised strings on both sides.

12. Section 11 — Training loop

☐

Optimizer: Adam, lr 3e-4
Scheduler: cosine annealing over `epochs`, or ReduceLROnPlateau on val CER
Loss: `nn.CTCLoss(blank=0, reduction="mean", zero_infinity=True)`
Model output must be `log_softmax` over the class dimension.
Gradient clipping at 5.0 (LSTMs on CTC are prone to spikes)

Per epoch:

☐

train: iterate, accumulate mean loss
val: decode greedily, compute CER and WER
log: epoch, train_loss, val_loss, val_cer, val_wer, lr, wall-clock seconds
save: checkpoint to Drive if val_cer improved
early stop after `early\_stop\_patience` epochs with no improvement

Track the count of infinite/NaN losses per epoch and print it. A non-zero count means the length constraint in 9.3 is being violated.

13. Section 12 — Evaluation and sample predictions



Load the best checkpoint.
Evaluate on the TEST split (clean labels).
Print final CER and WER.
Print a table of 15 random test lines: ground truth | prediction | per-line CER.
Display 5 line images inline with their predictions above them.
Plot two curves: train loss vs epoch, and val CER vs epoch.

The image display matters — it is the fastest way to spot a cropping bug that the metrics alone would hide.

14. Section 13 — Results logging

Append one row per run to `results/results.csv`. **This file is the paper.**

column	source
<code>run_name</code> , <code>timestamp</code>	
<code>dataset</code> , <code>data_fraction</code> , <code>n_train_lines</code> , <code>n_train_chars</code>	actual counts, not the requested fraction
<code>noise_type</code> , <code>noise_rate</code> , <code>noise_seed</code> , <code>frac_labels_actually_corrupted</code>	measured, not requested
<code>cnn_width_mult</code> , <code>rnn_hidden</code> , <code>n_params</code>	
<code>seed</code> , <code>batch_size</code> , <code>epochs_run</code> , <code>lr</code>	
<code>best_val_cer</code> , <code>test_cer</code> , <code>test_wer</code> , <code>best_epoch</code> , <code>train_seconds</code>	

Log **measured** quantities beside requested ones. If you asked for 20% noise and the injector delivered 19.4%, the plot must use 19.4%.

Open in append mode with a header written only if the file does not exist, so a Colab disconnect never destroys prior rows.

15. Section 14 — Sanity checks

Run these before trusting any result.

Check	Pass condition
Overfit test	Train on 32 lines, no noise, 200 epochs. Train CER must approach 0. If it cannot memorise 32 lines, the model, loss or decoding is broken.
Length constraint	Section 9.3 reports under ~1% violations.
Split disjointness	No <code>page_id</code> appears in more than one split.
Noise = 0 identity	<code>inject_noise(rate=0)</code> returns unchanged labels.
Determinism	Same config twice gives the same <code>test_cer</code> to 3 decimal places.
Vocabulary sanity	Printed charset contains no unexpected Latin or control characters.
Visual crop check	5 displayed line images each contain exactly one full line of text, not cut off.

The overfit test and the determinism test are the two that catch the most damage. Run them first.

16. What to run before the next meeting

Three runs, in this order:

#	Config	Purpose
1	<code>data_fraction=1.0, noise_type=None</code>	Baseline. Proves the pipeline works.
2	<code>data_fraction=0.25, noise_type=None</code>	First evidence that error responds to data size at all.
3	<code>data_fraction=1.0, noise_type=char_flip, noise_rate=0.2</code>	First noise data point.

Present: the three CER numbers, the results.csv, the sample-prediction table, and the actual line count extracted from the dataset.

Say plainly what is not yet done: no scaling curve is fitted from three points, and the dataset may be too small to fit one at all — the Devanagari portion is roughly 27,000 words, whereas fitting a power law wants

data size varied across orders of magnitude. Raising that as an open question is more valuable than presenting a curve fitted through too few points.

17. Known failure modes

Symptom	Likely cause
Parser returns zero lines	XML namespace not handled; use <code>{*}TextLine</code> or an explicit nsmap
Loss decreases but CER stays at 100%	Model emitting only blanks — usually too few epochs, or LR too high
Loss is <code>inf</code> or <code>nan</code>	CTC length constraint violated; see 9.3
CER above 100%	Predictions much longer than targets — check the duplicate-collapse step in decoding
Identical strings scoring as errors	Missing NFC normalisation on one side
Suspiciously low CER	Line-level rather than page-level split; leakage
Session dies mid-run	Checkpoint every epoch to Drive, not to local disk
Second epoch far slower than the first	Reading images from Drive; copy to <code>/content/</code> first

18. Dataset size limitation, and the migration path

FID4SA Devanagari holds roughly 27,000 words. Subsampling at 12.5 / 25 / 50 / 100 percent therefore spans **less than one order of magnitude** in data size. Four points across that narrow a range will produce a fitted line, but not a convincing power law, and a reviewer will say so.

Record the **actual extracted line and character counts** from Section 5 and report them. That measured number is the evidence for deciding whether to migrate.

If the project migrates to a larger corpus

The likely candidate is **IIIT-INDIC-HW** (IHTR 2023, IIIT Hyderabad): word images across ten Indic scripts — Bengali, Devanagari, Gujarati, Gurmukhi, Kannada, Odia, Malayalam, Tamil, Telugu, Urdu — with pre-built train/validation splits, distributed as JPG images with transcriptions in plain `train.txt` and `val.txt`.

Impact on this spec:

Section	Change
4, 5	Largely removed. No XML parsing, no cropping — images are pre-cropped and the labels are plain text. Replace with a simple manifest loader.
6	Unchanged. NFC normalisation and vocabulary building still apply.
6.5	Needs a replacement leakage guard. Word images have no <code>page_id</code> . If writer or source-document metadata exists, split on that. If it does not, say so explicitly in the paper rather than pretending a random split is clean.
7 onwards	Unchanged. Noise injection, model, training, metrics and logging all carry over as written.

The pipeline is deliberately built so that only the loading layer is dataset-specific. Keep it that way: nothing downstream of the manifest may reference XML, page geometry or a specific corpus.

Word images are shorter and easier than line images, and IIIT-INDIC-HW is handwritten rather than printed. Both differences must be stated, not glossed.

19. Order of implementation

Build and verify in this order. Do not write the training loop before the extractor is confirmed correct.

☐
Section 4 (inspect XML) -> confirm schema, then
Section 5 (extract lines) -> visually check 20 crops, then
Section 6 (vocab) -> read the charset, then
Section 7 (noise) + unit tests -> then
Sections 8-11 (model, training) -> overfit test on 32 lines, then
Sections 12-14 (eval, logging, checks) -> then the three runs in Section 16.

Data wrangling in Sections 4–6 will take longer than the model work. That is normal for archival material with skewed scans and inconsistent coordinate conventions. Budget accordingly.